claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-ad8d893c642d27524.jsonl`
- SHA-256: `e458e6be78aa23b71c443c80dae83da97d94b141b51af4ab6bdc64f422e0e3be`
- Source modified: `2026-06-13T19:13:53+00:00`
- Imported at: `2026-07-08T16:00:00+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:09:22.948Z)

Review backend/pipeline/segmenters/fusion_hybrid.py. Verify: net_crossings via rally_gate.gate_windows is correct; the person path lazily imports PersonDetector so the DEFAULT path never imports torch/cv2 through this module (check fusion_hybrid + segmenters/__init__ import chain ? does importing the registry pull torch?); detections_per_frame frame-range alignment with the shuttle trajectory `.frame`; substrate selection; _select_for_handoff gating logic (off-by-one, default 0 = no-op); the `family='fusion'` coupling to idx_cfg['fusion']['velocity_threshold']. Find correctness/cost/import bugs.

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:09:25.668Z)

I'll review the fusion implementation. Let me start by pulling the latest and reading the relevant files.

3. user (2026-06-13T19:09:29.021Z)

```
Already up to date.
---STATUS---
 M backend/config/models.py
 M backend/pipeline/detectors/person_detector.py
 M backend/pipeline/segmenters/__init__.py
 M backend/pipeline/segmenters/trajectory_hybrid.py
 M docs/README.md
 M tests/test_config.py
 M tests/test_trajectory_hybrid_equivalence.py
?? artifacts/
?? backend/pipeline/detectors/fusion_features.py
?? backend/pipeline/segmenters/fusion_hybrid.py
?? docs/QUALITY_ITERATIONS.md
?? scratch/HIGH_RISK_REMEDIATION_PLAN.md
?? scratch/PR1_BODY.md
?? scratch/PR1_HANDOFF.md
```

```
?? scratch/PR2_HANDOFF.md
?? scratch/PR2_deferred_hardening_status.patch
?? scratch/PR88_FIX.md
?? scratch/arial.ttf
?? scratch/debug_wsl.py
?? scratch/keep_awake.ps1
?? scratch/probe_cv2_count.py
?? scratch/probe_gemini.py
?? scratch/repro_drawtext_font.py
?? scratch/repro_movie_overlay.py
?? scratch/run_gx010125.py
?? scratch/stitch_min5.py
?? scratch/trim_video.py
?? tests/test_fusion_features.py
?? tests/test_fusion_hybrid.py
```

4. user (2026-06-13T19:09:29.036Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13        never imports torch / touches the GPU.
14      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17        **Persisted candidates remain the full superset** regardless, so the offline
experiment
18        harness can re-score any variant.
19
20      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24      experiment harness (EXPERIMENT_HARNESS.md), never silently.
25      """
26      from typing import Any, Dict, List, Optional, Tuple
27
28      from backend.pipeline.segmenters.base import segmenter_registry
29      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30      from backend.pipeline.detectors.base import DetectorRunner
31      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33      from backend.pipeline.detectors import rally_gate
34      from backend.pipeline.detectors import fusion_features as ff
35
36
37      class FusionSegmenter(TrajectoryHybridSegmenter):
38          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
```

```
40          family = "fusion"
41          display_label = "Fusion"
42
43          def __init__(self, db: Any, config: Any):
44              super().__init__(db, config)
45              # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46              self._person: Optional[Any] = None
47
48          @property
49          def name(self) -> str:
50              return "fusion_hybrid"
51
52          @property
53          def description(self) -> str:
54              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                      "AI provider sets semantic boundaries.")
57
58          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60              if substrate == "tracknet":
61                  cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62                  return TrackNetRunner(cfg), {
63                      "weights_path": cfg.weights_path,
64                      "queue_length": cfg.queue_length,
65                      "fusion_substrate": "tracknet",
66                  }
67              wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68              return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                      frame_width: float, video_path: str,
72                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73              stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74              # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75              # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76              # over the whole trajectory by rally_gate (camera-agnostic).
77              gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
78              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80              fcfg = idx_cfg.get("fusion", {})
81              if fcfg.get("cue_flags", {}).get("person", False):
82                  stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83              return stats
84
85          def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                               frame_width: float, video_path: str,
87                               fcfg: Dict[str, Any]) -> Dict[str, float]:
88              """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89              frame range and compute the scale/translation-invariant person features. Lazily
builds
90              ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91              # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
```

```
this module.
92              from backend.pipeline.detectors.person_detector import PersonDetector
93              if self._person is None:
94                  self._person = PersonDetector(self.config)
95
96              frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97              start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98              det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99              fw = det["frame_width"] or frame_width
100             fh = det["frame_height"] or 0.0
101
102             poly = fcfg.get("court_polygon")
103             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104             net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106             return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                               court_polygon=polygon, net=net)
108
109         def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                                 window_stats: List[Dict[str, Any]],
111                                 idx_cfg: Dict[str, Any]) -> List[int]:
112             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113             (0) keep every window ? identical to the substrate. Persisted candidates are
114             untouched (full superset)."""
115             fcfg = idx_cfg.get("fusion", {})
116             min_crossings = int(fcfg.get("min_crossings", 0))
117             min_players = int(fcfg.get("min_players", 0))
118             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120             keep: List[int] = []
121             for i, st in enumerate(window_stats):
122                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                     continue
124                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                     continue
126                 keep.append(i)
127             return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131
```

5. user (2026-06-13T19:09:29.039Z)

```
1       """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3       NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4       math is unit-testable in a GPU-free dev container. The detection (person boxes,
5       shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6       module only turns those signals into a handful of **scale/translation-invariant**
7       features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8       memorise this court/camera and fail to generalise off a small labelled set).
9
10      Conventions:
11      - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12      - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14      - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
```

```
    - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask, every
      detection counts ? the player-count-only mode).
    - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided = 0.0).

    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
    floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
    """
    from __future__ import annotations

    from typing import Dict, List, Optional, Sequence, Tuple

    BBox = Tuple[float, float, float, float]
    Net = Tuple[str, float]


    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]]) -> bool:
        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
        if polygon is None or len(polygon) < 3:
            return False
        x, y = point
        inside = False
        n = len(polygon)
        j = n - 1
        for i in range(n):
            xi, yi = polygon[i]
            xj, yj = polygon[j]
            # Does the horizontal ray at y cross edge (i, j)?
            if (yi > y) != (yj > y):
                x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
                if x < x_cross:
                    inside = not inside
            j = i
        return inside


    def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float, float]:
        """Normalised foot point (bottom-centre) of a person box ? where the player stands,
        the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, _y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)


    def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float, float]:
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)


    def person_in_court(box: BBox, frame_width: float, frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
        """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
        True (count everyone) when it is None."""
        if court_polygon is None:
            return True
        return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)

```

```
75
76      def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                          frame_height: float,
78                          court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79          """Per-frame count of in-court persons (one entry per sampled frame)."""
80          return [
81              sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
82              for _frame_idx, dets in sorted(per_frame.items())
83          ]
84
85
86      def _median(vals: Sequence[float]) -> float:
87          if not vals:
88              return 0.0
89          s = sorted(vals)
90          n = len(s)
91          return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94      def players_in_court(counts: Sequence[int]) -> float:
95          """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96          return _median(list(counts))
97
98
99      def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
101         if not counts:
102             return 0.0
103         return sum(1 for c in counts if c >= min_players) / len(counts)
104
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                            frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:
122             return 0.0
123         return sum(steps) / len(steps)
124
125
126     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                          frame_height: float, net: Optional[Net],
128                          court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
133         """
134         if net is None or not per_frame:
135             return 0.0
```

```
        axis, pos = net
        both = 0
        for _frame_idx, dets in per_frame.items():
            near, far = False, False
            for _tid, box in dets:
                if not person_in_court(box, frame_width, frame_height, court_polygon):
                    continue
                fx, fy = _foot_norm(box, frame_width, frame_height)
                coord = fx if axis == "x" else fy
                if coord < pos:
                    near = True
                else:
                    far = True
            if near and far:
                both += 1
        return both / len(per_frame)


    def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
                        pad_frac: float) -> bool:
        """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
        width/height (the 'adjacent / in-hand' tolerance)?"""
        x1, y1, x2, y2 = box
        px = pad_frac * frame_width
        py = pad_frac * frame_height
        return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)


    def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
                                  frame_width: float, frame_height: float,
                                  pad_frac: float = 0.03) -> float:
        """Fraction of (visible shuttle frames that were also person-sampled) in which the
        shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
        ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
        """
        if not per_frame or frame_width <= 0 or frame_height <= 0:
            return 0.0
        aligned = 0
        colocated = 0
        for p in shuttle_points or []:
            if not getattr(p, "visible", False):
                continue
            dets = per_frame.get(int(p.frame))
            if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
                continue
            aligned += 1
            if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
                   for _tid, box in dets):
                colocated += 1
        return colocated / aligned if aligned else 0.0


    # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
    # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
    PERSON_FEATURE_NAMES = [
        "players_in_court", "two_sided_motion", "mean_player_motion",
        "in_court_ratio", "shuttle_player_colocation",
    ]


    def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
```

```
197                                 frame_width: float, frame_height: float,
198                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                 net: Optional[Net] = None,
200                                 min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }
212
```

6. assistant (2026-06-13T19:09:31.590Z)

Now let me read the edited files and the supporting infrastructure to verify the import chain and the contracts.

7. user (2026-06-13T19:09:32.622Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20      import os
21      import time
22      import logging
23      import concurrent.futures
24      from typing import Any, Dict, List, Optional, Tuple
25
26      import cv2
27
28      from backend.pipeline.segmenters.base import BasePlaySegmenter
29      from backend.utils.video import get_video_duration, extract_video_chunk
30      from backend.sports import sport_registry
31      from backend.providers import provider_registry
32      from backend.pipeline.detectors.base import DetectorRunner
33
34      logger = logging.getLogger(__name__)
35
36
37      def _fmt_ts(seconds: float) -> str:
38          seconds = max(0, int(seconds))
```

```
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    class TrajectoryHybridSegmenter(BasePlaySegmenter):
        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""

        #: config section under `indexing` (velocity_threshold lives there), the
        #: output subdir for trajectories, and the metadata detector-tag prefix.
        family: str = ""
        #: human-readable label used in log lines.
        display_label: str = ""

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            """Return (runner, detector-specific detection_params extras)."""
            raise NotImplementedError

        @staticmethod
        def _probe_fps_width(video_path: str) -> tuple:
            """Return (fps, frame_width) for a video, with sensible fallbacks."""
            cap = cv2.VideoCapture(video_path)
            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
            cap.release()
            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

        @staticmethod
        def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
            """Provider-reported exchange count, else a duration-based estimate.

            One canonical implementation ? the twins had drifted (wasb relied on
            ``int(None)`` raising into the fallback; same result, by accident).
            """
            shot_count = segment.get("shuttle_exchange_count")
            if shot_count is None:
                return max(3, int(duration / 0.8))
            try:
                return int(shot_count)
            except (ValueError, TypeError):
                return max(3, int(duration / 0.8))

        # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
            returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
            rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
            features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
            return {
                "peak_velocity": cw["peak_velocity"],
                "mean_velocity": cw["mean_velocity"],
                "active_frame_count": cw["active_frame_count"],
                "track_id_count": cw["track_id_count"],
            }

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
```

```
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Indices of the candidate windows that proceed to the AI handoff (and thus may
            become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
            ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
            Persisted candidates are NOT affected ? they remain the full superset for
offline
            re-scoring."""
            return list(range(len(windows)))

        def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                          player_pool: Optional[List[str]] = None,
                          max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
            # override-ignore: the ABC declares the Result contract (Success|Failure)
            # but every live segmenter still returns a bare list ? the documented
            # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
            label = self.display_label
            # --- Sport profile + AI provider wiring (identical across segmenters) ---
            sport_name = self.config.get("sport", "badminton")
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            idx_cfg = self.config.get("indexing", {})
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))

            api_key_env_var = f"{provider_name.upper()}_API_KEY"
            api_key = os.environ.get(api_key_env_var)
            if not api_key:
                logger.error(
                    f"{api_key_env_var} environment variable is not set! "
                    f"To run the {provider_name} handoff, set your API key. "
                    f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
                )
                return []
            try:
                provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- Runner config + windowing thresholds ---
            runner, runner_params = self._build_runner(idx_cfg)

            velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": self.name,
```

```
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s WSL environment not ready: %s", label, msg)
                return []
            logger.info("%s WSL env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join("output", self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
            )
            logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                        label, time.time() - t_start, len(all_candidate_windows))

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector=self.name,
                        start_time=cw["start"], end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                        stats=window_stats[i], detection_params=detection_params,
```

```
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
            handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
            os.makedirs(handoff_dir, exist_ok=True)
            logger.info("Phase 3: %s validation on %d candidate windows...",
                        provider_name, len(handoff_indices))

            def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                w_start = max(0, window["start"] - handoff_padding)
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start
                w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                    return []
                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
                                                     label=f"Handoff {wi+1}")
                valid = []
                for seg in segments:
                    try:
                        seg["start_time"] = float(seg["start_time"]) + w_start
                        seg["end_time"] = float(seg["end_time"]) + w_start
                        seg["_candidate_id"] = candidate_ids[wi]
                        valid.append(seg)
                    except (ValueError, TypeError, KeyError) as e:
                        logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
                try:
                    os.remove(w_path)
                except OSError:
                    pass
                return valid

            ai_segments: List[dict] = []
            with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
                           for wi in handoff_indices]
                for future in futures:
                    ai_segments.extend(future.result())
            try:
                os.rmdir(handoff_dir)
            except OSError:
                pass
            logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
```

```
274                     dur = end - start
275                     metadata = {
276                         "shot_count": self._shot_count_from(segment, dur),
277                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                         "detector": f"{self.family}-hybrid-{model_name}",
279                     }
280                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281                     if segment_id:
282                         candidate_id = segment.get("_candidate_id")
283                         if candidate_id is not None:
284                             self.db.link_candidate_to_segment(candidate_id, segment_id)
285                         segments_data.append({
286                             "id": segment_id, "start_time": start, "end_time": end,
287                             "duration": dur, "metadata": metadata,
288                         })
289                 except Exception as e:
290                     logger.warning("Skipping invalid segment struct: %s", e)
291
292             return segments_data
293
```

8. user (2026-06-13T19:09:33.937Z)

```
warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be
replaced by CRLF the next time Git touches it
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index b2847c9..149282e 100644
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -78,6 +78,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         except (ValueError, TypeError):
             return max(3, int(duration / 0.8))

+    # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -------
+    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
+                                frame_width: float, video_path: str,
+                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
+        """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
+        returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
+        rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
+        features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
+        return {
+            "peak_velocity": cw["peak_velocity"],
+            "mean_velocity": cw["mean_velocity"],
+            "active_frame_count": cw["active_frame_count"],
+            "track_id_count": cw["track_id_count"],
+        }
+
+    def _select_for_handoff(self, windows: List[Dict[str, Any]],
+                            window_stats: List[Dict[str, Any]],
+                            idx_cfg: Dict[str, Any]) -> List[int]:
+        """Indices of the candidate windows that proceed to the AI handoff (and thus may
+        become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
+        ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
+        Persisted candidates are NOT affected ? they remain the full superset for offline
+        re-scoring."""
+        return list(range(len(windows)))
+
     def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                       player_pool: Optional[List[str]] = None,
                       max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
@@ -172,31 +197,36 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
```

```
                            f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                            f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

+        # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
+        # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+        # persistence and the handoff gate below.
+        window_stats = [
+            self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+            for cw in all_candidate_windows
+        ]
+
         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
         if store_candidates:
             for i, cw in enumerate(all_candidate_windows):
-                stats = {
-                    "peak_velocity": cw["peak_velocity"],
-                    "mean_velocity": cw["mean_velocity"],
-                    "active_frame_count": cw["active_frame_count"],
-                    "track_id_count": cw["track_id_count"],
-                }
                 candidate_ids[i] = self.db.add_candidate_segment(
                     video_id, detector=self.name,
                     start_time=cw["start"], end_time=cw["end"],
                     chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
-                    stats=stats, detection_params=detection_params,
+                    stats=window_stats[i], detection_params=detection_params,
                 )

         if not all_candidate_windows:
             return []

         # --- Phase 3: AI provider handoff over padded candidate clips ---
+        # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+        # candidates above stay the full superset ? only the served play_segments are gated.
+        handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
         os.makedirs(handoff_dir, exist_ok=True)
         logger.info("Phase 3: %s validation on %d candidate windows...",
-                    provider_name, len(all_candidate_windows))
+                    provider_name, len(handoff_indices))

         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
             w_start = max(0, window["start"] - handoff_padding)
@@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):

         ai_segments: List[dict] = []
         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
-            futures = [executor.submit(process_candidate_window, wi, cw)
-                       for wi, cw in enumerate(all_candidate_windows)]
+            futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
+                       for wi in handoff_indices]
             for future in futures:
                 ai_segments.extend(future.result())
         try:
```

9. assistant (2026-06-13T19:09:37.274Z)

Now let me look at the import chain (`segmenters/__init__.py`), the rally_gate's `gate_windows`, the person_detector diff, and the config models.

10. user (2026-06-13T19:09:39.537Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index bb56881..7f8191c 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -75,6 +75,39 @@ class WasbIndexConfig(BaseModel):
     min_free_gb: float = 0.0


+class FusionConfig(BaseModel):
+    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
+
+    Every default reproduces control behaviour: the fusion segmenter persists the
+    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
+    true ? the person-cue features, but it does NOT gate the served handoff unless a
+    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
+    is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);
+    supplied ? off-court persons (spectators / adjacent courts) are excluded.
+    """
+    # Which trajectory substrate seeds the windows (shuttle stays primary).
+    substrate: Literal["wasb", "tracknet"] = "wasb"
+    # Windowing velocity threshold for the fusion family (the engine reads
+    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
+    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
+    velocity_threshold: float = 0.02
+    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
+    # enabled ? so the default path never imports torch / touches the GPU.
+    cue_flags: dict[str, bool] = Field(default_factory=dict)
+    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
+    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-
scoring).
+    min_crossings: int = Field(default=0, ge=0)
+    # Served Gate B: when the person cue is on, drop windows with median in-court players
+    # < this. 0 = OFF.
+    min_players: int = Field(default=0, ge=0)
+    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
+    court_polygon: Optional[list[list[float]]] = None
+    # Net line for the person two-sided-motion split (person features only ? the shuttle
+    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
+    net_axis: Literal["x", "y"] = "y"
+    net_position: float = Field(default=0.5, ge=0.0, le=1.0)
+
+
 class IndexingConfig(BaseModel):
     default_segmenter: str = "yolo_hybrid"
     use_gpu: bool = True
@@ -106,6 +139,7 @@ class IndexingConfig(BaseModel):

     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
+    fusion: FusionConfig = Field(default_factory=FusionConfig)

     @field_validator("default_segmenter")
     @classmethod
diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
index 38ec2b3..5b931a4 100644
--- a/backend/pipeline/detectors/person_detector.py
+++ b/backend/pipeline/detectors/person_detector.py
@@ -252,3 +252,50 @@ class PersonDetector:
         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

         return windows
+
+    def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
+                             frame_skip: int = 3) -> Dict[str, Any]:
```

```
+        """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL video.
+
+        Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
+        ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards the
+        per-frame boxes), this returns the raw per-frame detections so the fusion features can
+        align them with the shuttle trajectory. It reads the source by **absolute frame index**
+        (seek + sequential read), NOT a re-extracted clip, so the returned frame indices match
+        the WASB trajectory's ``.frame`` exactly ? that alignment is what
+        ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
+
+        Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
+        "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
+        accuracy varies by codec (it lands near the requested keyframe then decodes forward);
+        treat the indices as ±a few frames on real footage ? fine for window-level features.
+        """
+        self.lazy_load_model()
+        cap = cv2.VideoCapture(video_path)
+        if not cap.isOpened():
+            logger.error(f"Could not open {video_path}")
+            return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
+
+        fps = cap.get(cv2.CAP_PROP_FPS)
+        if fps <= 0:
+            fps = 30.0
+        frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
+        frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
+        score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
+        step = max(frame_skip, 1)
+        tracker = self.make_tracker(fps / step)
+
+        start_frame = max(0, int(start_frame))
+        frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
+        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
+        frame_idx = start_frame
+        while frame_idx <= end_frame:
+            ret, frame = cap.read()
+            if not ret:
+                break
+            if (frame_idx - start_frame) % step == 0:
+                frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
+            frame_idx += 1
+        cap.release()
+        return {"frames": frames, "fps": fps,
+                "frame_width": float(frame_width), "frame_height": float(frame_height)}
diff --git a/backend/pipeline/segmenters/__init__.py b/backend/pipeline/segmenters/__init__.py
index c003d1a..fc5de77 100644
--- a/backend/pipeline/segmenters/__init__.py
+++ b/backend/pipeline/segmenters/__init__.py
@@ -4,6 +4,7 @@ from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
 from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
 from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
 from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
+from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter

 __all__ = [
     "segmenter_registry",
@@ -13,4 +14,5 @@ __all__ = [
     "HybridYoloSegmenter",
     "TrackNetHybridSegmenter",
     "WasbHybridSegmenter",
+    "FusionSegmenter",
 ]
```

11. user (2026-06-13T19:09:39.646Z)

```
    """Rally-start gate: suppress between-point false fires using shuttle net-crossings.

    WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
    the opponent for service after losing a point, that single trajectory looks like a
    rally. The discriminator is structural, not duration-based: a real rally has the
    shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
    single service feed crosses ~once.

    This module is camera-agnostic and needs NO new model ? it works on the trajectory
    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
    axis with the larger shuttle spread) and the net position (median shuttle coord on
    that axis ? the shuttle clusters at the two ends and crosses the net region often,
    so the median sits near the net), then counts sign changes of (coord - net) across
    the visible points inside a candidate window, with a deadband to ignore jitter.

    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
    windows from trajectory_to_action_windows, or to fold into it later.
    """
    from __future__ import annotations

    from dataclasses import dataclass
    from typing import List, Sequence, Tuple

    Interval = Tuple[float, float]


    @dataclass
    class GatedWindow:
        start: float
        end: float
        crossings: int
        visible_points: int
        kept: bool


    def _spread(vals: Sequence[float]) -> float:
        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
        if len(vals) < 2:
            return 0.0
        s = sorted(vals)
        lo = s[max(0, int(0.05 * (len(s) - 1)))]
        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
```

```
66          return "y", _median(ys), sy
67
68
69      def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70          """Count sign changes of (coord - net) across visible points, ignoring the
71          deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72          crossings = 0
73          last_side = None
74          for p in window_points:
75              if not p.visible:
76                  continue
77              v = (p.x if axis == "x" else p.y) - net
78              if abs(v) < deadband:
79                  continue
80              side = v > 0
81              if last_side is not None and side != last_side:
82                  crossings += 1
83              last_side = side
84          return crossings
85
86
87      def _points_in(points, fps: float, start: float, end: float):
88          return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91      def gate_windows(points, windows: List[Interval], fps: float,
92                       min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93          """Annotate each (start,end) window with its shuttle net-crossing count and a
94          keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95          axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96          axis, net, span = estimate_play_axis(points)
97          deadband = deadband_frac * span
98          out: List[GatedWindow] = []
99          for (s, e) in windows:
100             wp = _points_in(points, fps, s, e)
101             c = count_net_crossings(wp, axis, net, deadband)
102             vis = sum(1 for p in wp if p.visible)
103             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                                    kept=c >= min_crossings))
105         return out
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,
109                    min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110         """Convenience: return only the windows that pass the crossing gate."""
111         return [(g.start, g.end) for g in
112                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113
```

12. assistant (2026-06-13T19:09:43.479Z)

Now let me verify a few key contracts. Let me check `add_candidate_segment`'s `stats` handling, the `name` property usage for detector keying, the `PersonDetector.__init__` to confirm it accepts `self.config`, and whether `make_tracker`/`detect_and_track`/`lazy_load_model` exist.

13. user (2026-06-13T19:09:44.936Z)

```
1       """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3       Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4       the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5       `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6       next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
```

```
      later the learned classifier reads its persisted features. Keeping detection here (not
in
      the segmenter) is what makes "add a cue = register a producer" true.

      Behaviour is unchanged from the in-lined version ? only the home moved:
        - `lazy_load_model`               ? load a permissively-licensed torchvision detector
        - `make_tracker`                  ? a fresh supervision ByteTrack per chunk
        - `detect_and_track`              ? one frame ? confident persons with persistent IDs
        - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
          with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
          that feed the candidate `stats` JSON and the learned fusion features.

      Heavy deps (torchvision, supervision) are imported lazily inside the methods so
importing
      the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
      the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
      (ADR-005; ultralytics' AGPL YOLO was dropped).
      """
      import logging
      from typing import Any, Dict, List, Optional, Tuple

      import cv2
      import torch

      from backend.utils.geometry import get_velocity_normalised

      logger = logging.getLogger(__name__)

      # torchvision detection models are trained on COCO where the "person" category is
      # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
      # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
      _PERSON_CLASS_ID = 1

      # Permissively-licensed torchvision detectors selectable via config
      # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
      _DETECTOR_FACTORIES = {
          "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
          "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
          "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
      }
      _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"


      class PersonDetector:
          """Stage-1 person detection + tracking + motion-window extraction (a rally cue).

          Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
          ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
          min_action_window_duration}``); the model loads lazily on first use.
          """

          def __init__(self, config: Dict[str, Any]):
              self.config = config
              # Dynamic torchvision module (None until lazy_load_model); typed Any so the
              # `self.model([tensor])` inference call type-checks without a quarantine.
              self.model: Any = None
              self.device: Optional[str] = None

          def lazy_load_model(self) -> None:
              if self.model is not None:
                  return

              # Imported lazily so the module imports without torchvision present and so
              # the test suite (which pre-sets self.model) never pulls in model weights.
              from torchvision.models import detection as tv_detection
```

```
70
71              idx_cfg = self.config.get("indexing", {})
72              use_gpu = idx_cfg.get("use_gpu", True)
73              self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
74              if use_gpu and not torch.cuda.is_available():
75                  logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
76
77              model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
78              if model_name not in _DETECTOR_FACTORIES:
79                  logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
80                  model_name = _DEFAULT_DETECTOR
81
82              builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
83              logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
84              # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
85              self.model = builder(weights="DEFAULT")
86              self.model.eval()
87              self.model.to(self.device)
88
89          def make_tracker(self, fps: float) -> Any:
90              """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
91
92              Imported lazily so tests can mock this method without supervision installed.
93              """
94              import supervision as sv
95              frame_rate = max(1, int(round(fps)))
96              return sv.ByteTrack(frame_rate=frame_rate)
97
98          def detect_and_track(self, frame: Any, tracker: Any,
99                               score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100             """Detect persons in a frame and assign persistent track IDs.
101
102             Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103             This replaces the old ultralytics ``model.track(...)`` call, which bundled
104             detection AND association: torchvision does the detection, supervision's
105             ByteTrack does the association.
106             """
107             import supervision as sv
108
109             # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110             rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111             tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
112             with torch.no_grad():
113                 outputs = self.model([tensor])
114
115             out = outputs[0]
116             boxes = out["boxes"].detach().cpu().numpy()
117             labels = out["labels"].detach().cpu().numpy()
118             scores = out["scores"].detach().cpu().numpy()
119
120             # Keep only confident person detections.
```

14. user (2026-06-13T19:09:46.364Z)

```
56:    def __init__(self, config: Dict[str, Any]):
63:    def lazy_load_model(self) -> None:
89:    def make_tracker(self, fps: float) -> Any:
98:    def detect_and_track(self, frame: Any, tracker: Any,
```

15. assistant (2026-06-13T19:09:53.816Z)

Important finding: `person_detector.py` does `import cv2` and `import torch` at **module top level** (lines 26-27). So lazily importing `PersonDetector` does avoid torch ? but only because the import is deferred. Let me verify the full import chain: does importing the segmenter registry pull `trajectory_hybrid` (which imports `cv2` at top), and does `fusion_hybrid` pull anything heavy at module top.

`fusion_hybrid.py` imports at top: `wasb_runner`, `tracknet_runner`, `rally_gate`, `fusion_features`. Let me check whether `wasb_runner`/`tracknet_runner` import torch/cv2 at module top, since `trajectory_hybrid` already imports `cv2` at top anyway (so the registry already pulls cv2). The question is specifically about **torch**.

16. user (2026-06-13T19:09:55.746Z)

```
=== wasb_runner top imports ===
backend/pipeline/detectors/wasb_runner.py:12:import os
backend/pipeline/detectors/wasb_runner.py:13:import shlex
backend/pipeline/detectors/wasb_runner.py:14:import logging
backend/pipeline/detectors/wasb_runner.py:15:import subprocess
backend/pipeline/detectors/wasb_runner.py:16:from dataclasses import dataclass
backend/pipeline/detectors/wasb_runner.py:17:from typing import Dict, Any, Optional, Tuple
backend/pipeline/detectors/wasb_runner.py:21:from backend.pipeline.detectors.tracknet_runner
import to_wsl_mnt_path, TrackNetRunner
backend/pipeline/detectors/wasb_runner.py:22:from backend.pipeline.detectors.base import
DetectorRunner, WslCommandMixin, wsl_tilde_quote
```

17. user (2026-06-13T19:09:57.399Z)

```
=== tracknet_runner top imports ===
backend/pipeline/detectors/tracknet_runner.py:17:import os
backend/pipeline/detectors/tracknet_runner.py:18:import csv
backend/pipeline/detectors/tracknet_runner.py:19:import shlex
backend/pipeline/detectors/tracknet_runner.py:20:import logging
backend/pipeline/detectors/tracknet_runner.py:21:import subprocess
backend/pipeline/detectors/tracknet_runner.py:22:from dataclasses import dataclass
backend/pipeline/detectors/tracknet_runner.py:23:from typing import List, Dict, Any, Optional,
Tuple
backend/pipeline/detectors/tracknet_runner.py:25:from backend.pipeline.detectors.base import
DetectorRunner, WslCommandMixin, wsl_tilde_quote
=== fusion_features top imports ===
```

18. assistant (2026-06-13T19:10:02.211Z)

The runners don't import torch/cv2 at module top (they shell out to WSL). `fusion_features` imports nothing heavy. So the import chain for the registry pulls `cv2` (via `trajectory_hybrid` top-level import, pre-existing) but **not torch** ? torch only comes via `person_detector`, which `fusion_hybrid` imports lazily inside `_person_features`. The default path is torch-free. Let me confirm there's no eager `numpy` either, and verify the `name`/detector-keying and `add_candidate_segment` stats contract.

19. user (2026-06-13T19:10:03.765Z)

```
240:    def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
241-                              chunk_index: Optional[int] = None, confidence: Optional[float]
= None,
242-                              stats: Optional[Dict[str, Any]] = None,
243-                              detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
244-        """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
245-        detector-specific dicts stored as JSON. Returns the new candidate id, or None."""
246-        try:
247-            with self._connect() as conn:
248-                cursor = conn.cursor()
249-                cursor.execute(
```

```
250-                    """INSERT INTO candidate_segments
251-                       (video_id, detector, chunk_index, start_time, end_time, duration,
252-                        confidence, stats, detection_params)
253-                       VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
254-                    (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
255-                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
256-                )
257-                conn.commit()
258-                return cursor.lastrowid
259-        except Exception as e:
260-            logger.error(f"Failed to add candidate segment: {e}")
261-            return None
262-
263-    def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
264-        """Record that a candidate detection was confirmed as a given play_segment."""
265-        try:
266-            with self._connect() as conn:
267-                conn.cursor().execute(
268-                    "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
269-                    (segment_id, candidate_id)
270-                )
271-                conn.commit()
272-            return True
273-        except Exception as e:
274-            logger.error(f"Failed to link candidate {candidate_id} to segment {segment_id}:
{e}")
275-            return False
```

20. user (2026-06-13T19:10:04.702Z)

```
120             # Keep only confident person detections.
121             mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
122             if not mask.any():
123                 return []
124
125             # ByteTrack requires confidence to be populated (it filters on it internally).
126             dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
127             tracked = tracker.update_with_detections(dets)
128
129             result: List[Tuple[int, Tuple[float, float, float, float]]] = []
130             for i in range(len(tracked)):
131                 tid = tracked.tracker_id[i]
132                 if tid is None:
133                     continue
134                 x1, y1, x2, y2 = tracked.xyxy[i]
135                 result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
136             return result
137
138         def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
139                                               velocity_thresh: float, merge_gap: float,
140                                               frame_skip: int = 3,
141                                               chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
142             """
143             Runs person detection + tracking on a video chunk and extracts high-motion
144             action windows. Returns a list of dicts in the full video's timeframe, each
with:
145                 start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
146                 track_id_count, chunk_index.
147
148             Args:
149                 frame_skip: Process every Nth frame (1 = every frame). Higher values
```

```
                            dramatically reduce inference cost with minimal quality loss.
                chunk_index: Index of the source chunk (recorded on each window for
traceability).
            """
            cap = cv2.VideoCapture(chunk_path)
            if not cap.isOpened():
                logger.error(f"Could not open {chunk_path}")
                return []

            fps = cap.get(cv2.CAP_PROP_FPS)
            if fps <= 0:
                fps = 30.0

            frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
            if frame_width <= 0:
                frame_width = 1280.0  # Sensible fallback

            # Effective sampling rate after frame-skipping
            effective_fps = fps / max(frame_skip, 1)

            # A fresh tracker per chunk ? track IDs only need to be consistent within a
            # chunk (windows are computed per chunk, then merged across chunks by time).
            tracker = self.make_tracker(effective_fps)
            score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)

            # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
            # so windows can be enriched with motion stats for logging + fine-tuning.
            active_frames = []
            frame_idx = 0
            track_history: Dict[int, Tuple[float, float, float, float]] = {}

            while True:
                ret, frame = cap.read()
                if not ret:
                    break

                # Skip frames for performance ? at 30fps, every 3rd frame still
                # gives ~10 samples/second, plenty for human-scale velocity.
                if frame_idx % max(frame_skip, 1) != 0:
                    frame_idx += 1
                    continue

                # Detect persons + assign persistent track IDs.
                detections = self.detect_and_track(frame, tracker, score_thresh)

                frame_peak_velocity = 0.0
                frame_active_ids = set()
                current_ids = set()

                for track_id, bbox in detections:
                    current_ids.add(track_id)

                    if track_id in track_history:
                        prev_bbox = track_history[track_id]
                        velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
                        if velocity > velocity_thresh:
                            frame_peak_velocity = max(frame_peak_velocity, velocity)
                            frame_active_ids.add(track_id)

                    track_history[track_id] = bbox

                # Prune stale IDs that are no longer visible ? prevents false
                # velocity spikes when the tracker reassigns an old ID.
```

```
212                 track_history = {k: v for k, v in track_history.items() if k in current_ids}
213
214                 if frame_active_ids:
215                     active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
216
217                 frame_idx += 1
218
219             cap.release()
220
221             # Group active frames into continuous windows
222             if not active_frames:
223                 return []
224
225             max_gap_frames = int(merge_gap * fps)
226
227             def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
228                 velocities = [v for _, v, _ in group]
229                 track_ids = set().union(*[ids for _, _, ids in group])
230                 return {
231                     "start": group[0][0] / fps + chunk_start,
232                     "end": group[-1][0] / fps + chunk_start,
233                     "active_frame_count": len(group),
234                     "peak_velocity": max(velocities),
235                     "mean_velocity": sum(velocities) / len(velocities),
236                     "track_id_count": len(track_ids),
237                     "chunk_index": chunk_index,
238                 }
239
240             windows = []
241             group = [active_frames[0]]
242             for af in active_frames[1:]:
243                 if af[0] - group[-1][0] <= max_gap_frames:
244                     group.append(af)
245                 else:
246                     windows.append(_finalize(group))
247                     group = [af]
248             windows.append(_finalize(group))
249
250             # Filter out extremely short action windows
251             min_window_duration = self.config.get("indexing",
{}).get("min_action_window_duration", 2.0)
252             windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
253
254             return windows
```

21. assistant (2026-06-13T19:10:13.177Z)

Now I have a clear picture. Let me verify the specific points raised in the task.

1. `net_crossings` via `gate_windows` ? correctness. In `fusion_hybrid.py:77`, `rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps, min_crossings=0)` is called. `gate_windows` returns a list with one `GatedWindow`, and `.crossings` is read. `min_crossings=0` makes `kept` always True (irrelevant here since only `.crossings` is read). This is correct ? it counts net crossings for the window. Good.

2. `_select_for_handoff` gating ? default 0 = no-op. Default `min_crossings=0`, the guard is `if min_crossings > 0 and ...`, so 0 disables it. Correct. Let me check the off-by-one: `st.get("net_crossings", 0.0) < min_crossings` ? a window with exactly `min_crossings` crossings is kept (>=). With `min_crossings=2`, a window with 2 crossings passes. That matches `rally_gate.apply_gate`'s `kept = c >= min_crossings` semantics. Consistent.

3. The `players_in_court` key used in gating. `_select_for_handoff:124` reads `st.get("players_in_court", 0.0)`. `compute_person_features` returns `players_in_court`. Good ? key matches.

Now let me examine a potential issue in `detections_per_frame` frame alignment and the `frame_skip` config default mismatch.

22. user (2026-06-13T19:10:14.759Z)

```
backend\config\models.py:122:    yolo_frame_skip: int = 3
backend\pipeline\segmenters\fusion_hybrid.py:96:        frame_skip = self.config.get("indexing",
{}).get("yolo_frame_skip", 3)
backend\pipeline\segmenters\fusion_hybrid.py:98:        det =
self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
backend\pipeline\detectors\person_detector.py:140:
frame_skip: int = 3,
backend\pipeline\detectors\person_detector.py:149:            frame_skip: Process every Nth
frame (1 = every frame). Higher values
backend\pipeline\detectors\person_detector.py:167:        effective_fps = fps / max(frame_skip,
1)
backend\pipeline\detectors\person_detector.py:187:            if frame_idx % max(frame_skip, 1)
!= 0:
backend\pipeline\detectors\person_detector.py:257:                                frame_skip: int =
3) -> Dict[str, Any]:
backend\pipeline\detectors\person_detector.py:285:        step = max(frame_skip, 1)
backend\pipeline\segmenters\yolo_hybrid.py:101:        frame_skip =
idx_cfg.get("yolo_frame_skip", 3)
backend\pipeline\segmenters\yolo_hybrid.py:113:            "frame_skip": frame_skip,
backend\pipeline\segmenters\yolo_hybrid.py:172:                    frame_skip=frame_skip,
chunk_index=ci
backend\pipeline\segmenters\yolo_hybrid.py:197:                    frame_skip=frame_skip,
chunk_index=ci
```

23. user (2026-06-13T19:10:16.448Z)

```
80:class TrajectoryPoint:
81:    frame: int
248:        points.sort(key=lambda p: p.frame)
284:                dframes = p.frame - prev.frame
289:                        active.append((p.frame, velocity))
```

24. assistant (2026-06-13T19:10:23.169Z)

Now let me verify the critical frame-range alignment issue. In `_person_features` (fusion_hybrid.py:97), `start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)`. These are frame indices computed from window seconds × fps. The shuttle trajectory `.frame` is the original-video frame index.

But there's a subtle issue: `detections_per_frame` samples frames at `(frame_idx - start_frame) % step == 0`, i.e. frames `start_f, start_f+step, start_f+2*step, ...`. The shuttle trajectory `.frame` values are produced by WASB at whatever sampling WASB uses (likely every frame). So `shuttle_player_colocation` aligns a shuttle point at frame N only when `per_frame.get(int(p.frame))` exists ? i.e., only when N happens to be a sampled person frame. Let me check the `_points_in`/colocation alignment more carefully and confirm `shuttle_in` uses `<=` consistently.

Let me look at the TrajectoryPoint definition and how WASB frames are numbered, plus check the test files to understand expected behavior.

25. user (2026-06-13T19:10:24.123Z)

```
78
79      @dataclass
80      class TrajectoryPoint:
81          frame: int
82          visible: bool
83          x: float
84          y: float
```

```
85
86
87      class TrackNetRunner(WslCommandMixin, DetectorRunner):
88          """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90          Implements the common DetectorRunner contract so a future Linux-native or
91          alternative trajectory runner can be substituted without touching the hybrids.
92          """
93
94          def __init__(self, cfg: TrackNetConfig):
95              self.cfg = cfg
96
97          def healthcheck(self) -> Tuple[bool, str]:
```

26. user (2026-06-13T19:10:24.545Z)

```
1       """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3       Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4       keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5       substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6       persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7       """
8       from types import SimpleNamespace
9       from unittest.mock import MagicMock, patch
10
11      from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12      from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14      # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15      # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16      WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17               "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18      WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19               "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22      def _points():
23          pts = []
24          for f in range(30, 120):       # window A frames: y flips 100<->900 around the net
(~500)
25              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26          for f in range(300, 360):      # window B frames: y == net level -> deadband -> no
crossings
27              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
28          return pts
29
30
31      def _runner():
32          r = MagicMock()
33          r.healthcheck.return_value = (True, "env ok")
34          r.run_predict.return_value = "out.csv"
35          r.parse_trajectory_csv.return_value = _points()
36          r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
37          return r
38
39
40      def _run(indexing=None, provider_segments=None):
41          provider = MagicMock()
42          provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
```

```
        db = MagicMock()
        db.add_candidate_segment.side_effect = [101, 102, 103, 104]
        db.add_play_segment.return_value = 200

        with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
             patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)),
\
             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
             patch("os.environ.get", return_value="dummy_key"):
            preg.get.return_value = provider
            seg = FusionSegmenter(db, {"indexing": indexing or {}})
            res = seg.process_video("v1", "clip.mp4")
            return db, provider, res


    def _stats_calls(db):
        """Persisted stats dicts, in call order."""
        return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]


    def test_net_crossings_persisted_and_person_off_by_default():
        db, _provider, _res = _run()
        stats = _stats_calls(db)
        assert len(stats) == 2                              # both windows persisted
(superset)
        assert all("net_crossings" in s for s in stats)
        # Window A (rally) crosses many times; window B (held) zero.
        assert stats[0]["net_crossings"] >= 2
        assert stats[1]["net_crossings"] == 0.0
        # Person cue OFF by default -> NONE of the person features present.
        assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
        # The 4 base motion keys are still there.
        assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
                   for s in stats)


    def test_persisted_candidates_are_superset_of_substrate_windows():
        db, _provider, _res = _run()
        spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
        assert spans == [(1.0, 4.0), (10.0, 12.0)]          # exactly the substrate windows,
unfiltered


    def test_default_hands_off_all_windows():
        _db, provider, _res = _run()                        # min_crossings default 0 -> no
gating
        assert provider.analyze_video.call_count == 2


    def test_gate_a_drops_low_crossing_window_from_handoff_only():
        db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
        # Persistence is still the full superset (offline substrate intact)...
        assert len(_stats_calls(db)) == 2
        # ...but only the high-crossing window A reaches the AI handoff.
        assert provider.analyze_video.call_count == 1


```

```
98      def test_person_cue_adds_features_when_enabled():
99          fake_pd = MagicMock()
100         fake_pd.detections_per_frame.return_value = {
101             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
102             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
103         }
104         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
return_value=fake_pd):
105             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
106         stats = _stats_calls(db)
107         assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
108         assert all("net_crossings" in s for s in stats)
109         fake_pd.detections_per_frame.assert_called()         # the person detector WAS used
110
```

27. assistant (2026-06-13T19:10:33.409Z)

Now I have a complete picture. Let me verify one more thing ? the `_points_in` deadband edge case in the test (window B at net level) and double-check whether `gate_windows` recomputes `estimate_play_axis` per-call (cost concern, since it's called once per window over the whole trajectory).

In `_enrich_candidate_stats:77`, `gate_windows(points, [(cw["start"], cw["end"])], fps, min_crossings=0)` is called **once per window**, and `gate_windows` internally calls `estimate_play_axis(points)` over the **entire trajectory** every time (rally_gate.py:96). And `_points_in` (rally_gate.py:88) also scans **all** points per window. So for W windows and P trajectory points, this is O(W·P) plus W full sorts of all visible coords inside `estimate_play_axis` ? `_spread` sorts all xs and ys. This is a real but minor cost concern. Let me confirm there's no shared precompute.

Let me also confirm the frame-range alignment concern in `shuttle_player_colocation` is real by checking step sampling vs WASB frame numbering, and check `two_sided_motion` over the sampled `per_frame`.

Let me verify the WASB CSV frames are per-frame (step 1) so the `% step` sampling creates a sparse alignment.

28. user (2026-06-13T19:10:36.151Z)

```
backend/pipeline/detectors/tracknet_runner.py:240:
points.append(TrajectoryPoint(
=== parse_trajectory_csv ===
backend/pipeline/detectors/tracknet_runner.py:233:    def parse_trajectory_csv(csv_win_path:
str) -> List[TrajectoryPoint]:
```

29. user (2026-06-13T19:10:36.924Z)

```
228             return dest
229
230         # ------------------------------------------------------------- CSV -> windows
231
232         @staticmethod
233         def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
234             """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
235             points: List[TrajectoryPoint] = []
236             with open(csv_win_path, newline="") as f:
237                 reader = csv.DictReader(f)
238                 for row in reader:
239                     try:
240                         points.append(TrajectoryPoint(
241                             frame=int(row["Frame"]),
242                             visible=int(row["Visibility"]) == 1,
243                             x=float(row["X"]),
244                             y=float(row["Y"]),
```

```
                        ))
                    except (KeyError, ValueError):
                        continue
            points.sort(key=lambda p: p.frame)
            return points

        @staticmethod
        def trajectory_to_action_windows(
            points: List[TrajectoryPoint],
            fps: float,
            frame_width: float,
            chunk_start: float = 0.0,
            velocity_thresh: float = 0.02,
            merge_gap: float = 2.0,
            min_window_duration: float = 2.0,
            chunk_index: Optional[int] = None,
        ) -> List[Dict[str, Any]]:
            """Convert a shuttle trajectory into candidate rally windows.

            A frame is "active" when the shuttle is visible AND its inter-frame
            displacement (normalised by frame width, per second) exceeds
            ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
            ``merge_gap`` seconds of each other are grouped into one window; short
            windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
            shape feeding the AI-handoff phase is identical.
            """
            if fps <= 0:
                fps = 30.0
            if frame_width <= 0:
                frame_width = 1280.0

            # Compute per-frame normalised velocity from the last *visible* point.
            active = []  # (frame_idx, velocity)
            prev: Optional[TrajectoryPoint] = None
            for p in points:
                if not p.visible:
                    prev = None  # break the trajectory; absent shuttle = not in flight
                    continue
                if prev is not None:
                    dframes = p.frame - prev.frame
                    if dframes > 0:
                        dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
                        velocity = (dist / frame_width) * (fps / dframes)
                        if velocity > velocity_thresh:
                            active.append((p.frame, velocity))
```

30. assistant (2026-06-13T19:11:05.251Z)

I now have everything I need. The WASB/TrackNet CSV produces one `TrajectoryPoint` per frame (Frame column is per-frame). The person detector samples every `step`-th frame. So `shuttle_player_colocation` aligns only on the subset of shuttle frames that coincide with sampled person frames ? which is by design (it skips unaligned frames via `dets is None`), and the docstring documents this. Not a bug.

Let me do a final check on the `int(cw["start"] * fps)` truncation vs the rally_gate `_points_in` which uses `start * fps <= p.frame` (no truncation). This is a subtle inconsistency: the person path's `shuttle_in` filter uses truncated integer frame bounds (`start_f <= p.frame <= end_f`), while the same window's net_crossings uses float `start*fps <= frame`. Minor. Let me verify the test for fusion_features and confirm `players_in_court` returns a float median (used in gating as `< min_players`).

The median of `[2,2,2]` = 2.0 (float), and gating `2.0 < 2` is False ? kept. Correct.

Let me run the two new test files to confirm they pass (pure logic, no GPU).

31. user (2026-06-13T19:11:13.134Z)

```
..............                                                           [100%]
14 passed in 5.72s
```

32. assistant (2026-06-13T19:11:17.176Z)

Tests pass. Now let me verify the one cost concern empirically ? that `gate_windows` recomputes `estimate_play_axis` over the whole trajectory per window ? and confirm the import chain claim (registry import is torch-free) with an actual import check, since the dev container lacks torch.

33. user (2026-06-13T19:11:27.231Z)

```
torch in sys.modules after registry import: True
torchvision in sys.modules: False
cv2 in sys.modules: True
fusion_hybrid registered: n/a
```

34. assistant (2026-06-13T19:11:30.788Z)

Interesting ? `torch` IS in `sys.modules` after importing the segmenter registry. Let me find out what pulls torch eagerly, since the dev container apparently has torch installed here. The claim in the code comments is that the default path never imports torch through `fusion_hybrid`. Let me trace what imports torch eagerly in the registry chain.

35. user (2026-06-13T19:11:39.994Z)

```
After importing fusion_hybrid module ALONE:
  torch loaded: True
  torchvision loaded: False
  person_detector loaded: True
```

36. assistant (2026-06-13T19:11:44.086Z)

This is a significant finding. Importing `fusion_hybrid` **alone** pulls both `torch` and `person_detector` into `sys.modules` ? the lazy-import claim is violated. Let me find the culprit. `fusion_hybrid.py` top-level imports `wasb_runner`, `tracknet_runner`, `rally_gate`, `fusion_features`. One of these must transitively import `person_detector` (which does `import torch` at top, line 27). Let me trace.

37. user (2026-06-13T19:11:47.329Z)

```
backend.pipeline.detectors.wasb_runner -> torch: False | person_detector: False
backend.pipeline.detectors.tracknet_runner -> torch: False | person_detector: False
backend.pipeline.detectors.rally_gate -> torch: False | person_detector: False
backend.pipeline.detectors.fusion_features -> torch: False | person_detector: False
```

38. assistant (2026-06-13T19:11:50.440Z)

None of `fusion_hybrid`'s direct imports pull torch. So the earlier result must have been contaminated by import order in the same interpreter. Let me re-test `fusion_hybrid` truly in isolation in a fresh subprocess.

39. user (2026-06-13T19:12:07.615Z)

```
FRESH interpreter, fusion_hybrid alone:
  torch loaded: True
  person_detector loaded: True
=== now the registry package alone ===
FRESH interpreter, segmenters package:
  torch loaded: True
  person_detector loaded: True
```

```
  yolo_hybrid imports person_detector at top: True
```

40. assistant (2026-06-13T19:12:16.851Z)

```
Now the real cause is clear. The `__init__.py` for the segmenters package imports `yolo_hybrid`
(line 4 of the diff context shows `from backend.pipeline.segmenters.yolo_hybrid import
HybridYoloSegmenter`), and `yolo_hybrid` imports `PersonDetector` at module top ?
`person_detector` ? `import torch`. So `import backend.pipeline.segmenters` already pulls torch
**regardless of fusion_hybrid** ? this is pre-existing behavior from `yolo_hybrid`.

But when importing `fusion_hybrid` *as a submodule path*
`backend.pipeline.segmenters.fusion_hybrid`, Python first executes the package `__init__.py`,
which imports `yolo_hybrid` ? torch. That's why even "fusion_hybrid alone" pulls torch. The
fusion module *itself* does not. Let me confirm by importing the fusion module file directly
bypassing the package init isn't really possible cleanly, but the per-import trace above already
proved fusion_hybrid's own imports are torch-free.

So the lazy-import claim in `fusion_hybrid.py` docstring ("the default path never imports torch
/ touches the GPU") is **accurate about the fusion module's own design** but **misleading about
reality**: the registry already eagerly imports torch via `yolo_hybrid`. This is a pre-existing
condition, not introduced by this change, but it means the stated benefit ("importing the
registry pull torch?" ? the task's explicit question) is: **YES, the registry pulls torch, but
via yolo_hybrid, not via fusion_hybrid.** The fusion change neither adds nor removes that.

Let me confirm yolo_hybrid's top-level import to nail this down precisely.
```

41. user (2026-06-13T19:12:18.415Z)

```
5:(`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
18:import os
19:import time
20:import logging
21:import concurrent.futures
22:from typing import List, Dict, Any, Tuple
24:from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
25:from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
26:from backend.utils.video import get_video_duration, extract_video_chunk
27:from backend.sports import sport_registry
28:from backend.providers import provider_registry
46:        self.person = PersonDetector(config)
```

42. assistant (2026-06-13T19:12:36.558Z)

```
Confirmed. `yolo_hybrid.py:25` imports `PersonDetector` at module top, and
`person_detector.py:27` does `import torch` at top. So the segmenters package `__init__.py` has
*always* pulled torch via `yolo_hybrid` ? this predates the fusion change.

The fusion module's own lazy-import discipline is correct and verified (its 4 direct imports are
torch-free). The docstring claim is about `fusion_hybrid` not adding torch to the path, which is
true. But the broader framing ("the default path never imports torch") is undermined by the pre-
existing `yolo_hybrid` eager import ? worth flagging as a doc-accuracy issue, low severity, and
clearly not introduced here.

Now let me verify the cost concern in `gate_windows` is real (per-window recompute of
axis/spread over the whole trajectory), and check one correctness subtlety in
`count_net_crossings`: the deadband is computed from the **whole-trajectory** span, and a window
where the shuttle sits exactly at net level yields 0 crossings (the test confirms WIN_B = 0).
Good.

Let me look at whether there's an off-by-one or float-truncation mismatch between the person
path's frame bounds and the trajectory frame numbering, and confirm `shuttle_in` is actually
used (it is passed to `compute_person_features`). Let me re-read lines 96-107 of fusion_hybrid
once more for the `shuttle_in` and frame bounds.
```

The frame bounds: `start_f, end_f = int(cw["start"]*fps), int(cw["end"]*fps)` (line 97). These are passed to `detections_per_frame(video_path, start_f, end_f, frame_skip)`, and `shuttle_in = [p for p in points if start_f <= p.frame <= end_f]`. Both person sampling and shuttle filtering use the same integer bounds ? internally consistent. The `net_crossings` for the same window uses rally_gate's float bounds (`start*fps <= frame`). For `int()` truncation, e.g. start=1.0,fps=30 ? 30 both ways; end=4.0 ? 120 both. With non-integer window times these differ by <1 frame. Negligible. Not a real bug.

Let me check one last potential issue: `detections_per_frame` returns `fps` but the fusion path ignores it (uses the window's `fps` from `_probe_fps_width`). And `det["frame_width"] or frame_width` ? if `frame_width` is 0.0 (cap couldn't read), falls back to probe width. Fine. Let me verify `make_tracker(fps/step)` ? fps here is the cap's native fps, divided by step. Consistent with `extract_action_windows_from_chunk` which uses `effective_fps = fps/max(frame_skip,1)`. Good.

Now let me confirm the `family='fusion'` ? `velocity_threshold` coupling. In `trajectory_hybrid.py:149`: `velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)`. For FusionSegmenter, `self.family = "fusion"`, so it reads `idx_cfg["fusion"]["velocity_threshold"]`. The `FusionConfig` has `velocity_threshold: float = 0.02`. Coupling is correct and the config key exists.

But there's a subtle config-shape concern: `_build_runner` reads `idx_cfg.get("fusion", {}).get("substrate", "wasb")` and the WASB/TrackNet runner configs are built from `idx_cfg` via `WasbConfig.from_indexing_cfg(idx_cfg)` ? which reads `idx_cfg["wasb"]`, NOT `idx_cfg["fusion"]`. So the fusion segmenter uses `fusion.velocity_threshold` for windowing but `wasb.*` for the runner weights/params. Let me verify that's intended and that `from_indexing_cfg` reads the right section.

43. user (2026-06-13T19:12:38.226Z)

```
backend/pipeline/detectors/tracknet_runner.py:46:    def from_indexing_cfg(cls, idx_cfg:
Dict[str, Any]) -> "TrackNetConfig":
backend/pipeline/detectors/wasb_runner.py:45:    def from_indexing_cfg(cls, idx_cfg: Dict[str,
Any]) -> "WasbConfig":
```

44. user (2026-06-13T19:12:38.998Z)

```
24      logger = logging.getLogger(__name__)
25
26      # Windows path to the inference wrapper we stage into WSL.
27      _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30      @dataclass
31      class WasbConfig:
32          """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/WASB-SBDT"
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "wasb"
37          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38          sport: str = "badminton"
39          wsl_stage_dir: str = "~/clips"
40          timeout_sec: int = 1800  # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41          keep_frames: bool = False  # retain staged video + frame cache after success (debug
only)
42          min_free_gb: float = 0.0   # warn if WSL free space below this before a run (0 =
off)
43
44          @classmethod
45          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
46              w = (idx_cfg or {}).get("wasb", {}) or {}
47              return cls(
```

```
48                  distro=w.get("wsl_distro", "Ubuntu"),
49                  repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
50                  conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
51                  conda_env=w.get("conda_env", "wasb"),
52                  weights_path=w.get("weights_path",
53                                     "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
54                  sport=w.get("sport", "badminton"),
55                  wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
56                  timeout_sec=int(w.get("timeout_sec", 1800)),
57                  keep_frames=bool(w.get("keep_frames", False)),
58                  min_free_gb=float(w.get("min_free_gb", 0.0)),
59              )
60
61
62      class WasbRunner(WslCommandMixin, DetectorRunner):
63          def __init__(self, cfg: WasbConfig):
64              self.cfg = cfg
65
66          def healthcheck(self) -> Tuple[bool, str]:
67              """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
68              cmd = (f"conda activate {self.cfg.conda_env} && "
69                     f"python -c \"import torch; print('torch', torch.__version__, "
70                     f"'cuda', torch.cuda.is_available())\"")
71              try:
72                  res = self._wsl_bash(cmd)
73              except FileNotFoundError:
74                  return False, "`wsl` not found ? Windows + WSL2 required."
75              except subprocess.TimeoutExpired:
76                  return False, "WSL healthcheck timed out."
77              out = (res.stdout or "").strip()
78              if res.returncode != 0:
79                  return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
80              return True, out
81
82          def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
83              src_mnt = to_wsl_mnt_path(video_win_path)
```

45. assistant (2026-06-13T19:12:54.437Z)

This confirms a real config-design inconsistency: `FusionSegmenter` reads its **windowing** velocity threshold from `idx_cfg["fusion"]["velocity_threshold"]` (via `family="fusion"`) but reads its **substrate runner config** (weights, conda env, WSL distro, etc.) from `idx_cfg["wasb"]` (or `["tracknet"]`) via `from_indexing_cfg`.

This is defensible (substrate config is shared with the standalone wasb/tracknet segmenters) and the docstrings hint at it ("set equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run"). But it's a sharp edge: a user who tunes `indexing.wasb.velocity_threshold` and then runs `fusion_hybrid` will silently get the **default 0.02** windowing (from `fusion.velocity_threshold`), NOT their tuned wasb value ? so fusion's candidate windows can differ from the wasb run it's meant to A/B against, unless the user also mirrors the threshold into `fusion`. The default-reproduces-substrate claim (docstring lines 20-22: "Defaults reproduce the substrate exactly") holds only when `wasb.velocity_threshold` is at its default. This is a med-severity correctness/usability trap worth documenting or reconciling.

Let me verify the `_select_for_handoff` empty-keep edge: if gating drops ALL windows, `handoff_indices=[]`, then Phase 3 submits zero futures, `ai_segments` empty, and the function returns `[]`. The `if not all_candidate_windows: return []` guard (line 219) is *before* gating, so persistence still happens. No crash. Good.

Now let me check the cost claim definitively: `gate_windows` re-runs `estimate_play_axis` (which sorts all visible xs and ys) once per window. Let me confirm there's no caching and quantify.

`fusion_hybrid.py:77` calls `gate_windows(points, [(cw_single_window)], ...)` inside `_enrich_candidate_stats`, which runs once per window via the list comprehension in

`trajectory_hybrid.py:203-206`. Each `gate_windows` call ? `estimate_play_axis(points)` ? two `_spread` calls each sorting all visible coords (O(P log P)), plus `_points_in` scanning all P points. So total = O(W · P log P). For a long video this is wasteful but not catastrophic (it's GPU-free pure Python over a few-thousand-point trajectory). The cleaner design would estimate axis/net ONCE and pass per-window, or call `gate_windows(points, all_windows, ...)` once. Low-severity cost issue.

I have enough to finalize. Let me run the broader equivalence test that was edited to make sure the base-class refactor didn't break the twins.

46. user (2026-06-13T19:13:03.296Z)

```
.........................                                                [100%]
28 passed in 5.63s
```

47. assistant (2026-06-13T19:13:10.960Z)

All tests pass. I have a complete and verified picture. Let me compile the findings.

Summary of what I verified:
- **net_crossings via gate_windows**: correct (`.crossings` read with `min_crossings=0`, single-window list).
- **Lazy PersonDetector import**: the fusion module's *own* 4 imports are torch-free (traced in isolation). BUT the segmenters package `__init__.py` already pulls torch eagerly via `yolo_hybrid` ? `person_detector` ? `import torch` ? pre-existing, not introduced here. The docstring's broad "default path never imports torch" is misleading at the registry level.
- **detections_per_frame alignment**: frame indices are original-video absolute (seek + sequential read), matching trajectory `.frame`; consistent integer bounds on both person sampling and shuttle filtering.
- **substrate selection**: correct; reads `fusion.substrate`.
- **_select_for_handoff gating**: default 0 = no-op (correct), `>=` semantics consistent with rally_gate, empty-keep doesn't crash.
- **family='fusion' coupling**: reads `fusion.velocity_threshold` for windowing but `wasb.*`/`tracknet.*` for runner config ? a real split-config trap.

48. user (2026-06-13T19:13:34.396Z)

Structured output provided successfully

49. assistant (2026-06-13T19:13:53.101Z)

I have completed the review. Here is a summary of my findings.

Review of fusion P2 (`fusion_hybrid.py`)

I verified each point raised and confirmed the implementation is largely correct. The pure feature math, net-crossing gating semantics, substrate selection, and frame alignment all check out. All 56 relevant tests pass (`test_fusion_hybrid`, `test_fusion_features`, `test_trajectory_hybrid_equivalence`, `test_config`).

What's correct (verified)
- **`net_crossings` via `gate_windows`** (`fusion_hybrid.py:77`): correct ? reads `.crossings` from the single-window result with `min_crossings=0` (so `kept` is irrelevant).
- **`_select_for_handoff` gating** (`fusion_hybrid.py:109-127`): default `min_crossings=0`/`min_players=0` is a true no-op (`if min_crossings > 0 and ...`). The `<` comparison is consistent with `rally_gate.apply_gate`'s `c >= min_crossings`. Dropping all windows doesn't crash (empty `handoff_indices` ? returns `[]`), and the `players_in_court` key matches `compute_person_features`'s output.
- **`detections_per_frame` alignment** (`person_detector.py:255-301`): reads by absolute frame index (seek + sequential), so indices match the WASB trajectory `.frame`. Person sampling and `shuttle_in` filtering use the same integer bounds.
- **substrate selection** (`fusion_hybrid.py:58-68`): correct, reads `fusion.substrate`.
- **Fusion module's own imports are torch-free**: traced in isolation ? its 4 top-level imports (`wasb_runner`, `tracknet_runner`, `rally_gate`, `fusion_features`) do not pull torch;

`PersonDetector` is correctly imported lazily inside `_person_features`.

Issues found (ranked)

1. **MED ? split config sections for `family='fusion'`** (`fusion_hybrid.py:58-68` + `trajectory_hybrid.py:149`): windowing `velocity_threshold` is read from `idx_cfg['fusion']`, but the substrate runner config (weights, conda, WSL) is read from `idx_cfg['wasb']`/`['tracknet']` via `from_indexing_cfg`. A user who tunes `indexing.wasb.velocity_threshold` and runs `fusion_hybrid` silently gets the default `0.02` windowing, producing a *different* candidate set than the wasb run it's meant to A/B against. The "Defaults reproduce the substrate exactly" docstring only holds when `wasb.velocity_threshold` is also at default.

2. **LOW ? docstring overstates the torch-free guarantee**: importing the registry *does* pull torch in a fresh interpreter, but via `yolo_hybrid.py:25` (eager `PersonDetector` import ? `person_detector.py:27` eager `import torch`), **not** via `fusion_hybrid`. Pre-existing, not introduced by this PR. The fusion docstring's "the default path never imports torch" claim is misleading at the registry level ? it's only true of this module.

3. **LOW ? cost: `gate_windows` re-estimates play-axis per window** (`fusion_hybrid.py:77` + `rally_gate.py:91-105`): called once per window, and each call re-runs `estimate_play_axis` (two full sorts of all visible coords) over the whole trajectory, despite axis/net being window-independent. O(W·P log P) redundant pure-Python work. Estimate once per video instead.

Relevant files: `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/fusion_hybrid.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/fusion_features.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/trajectory_hybrid.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/person_detector.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/rally_gate.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/yolo_hybrid.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/models.py`.